

Sofa Project

Deep Analysis Report

Comprehensive Bug Analysis, Security Audit, and Architecture Review

Repository: <https://github.com/fahimahamedwork/sofa>

Date: May 17, 2026

Analyzed by: Z.ai Automated Code Analysis

Executive Summary: The Sofa project is a self-hosted PaaS panel built with Go (backend) and React/TypeScript (frontend). While the project demonstrates a solid architectural vision with clean separation of concerns, it contains **51 identified issues** across security, functionality, API design, and infrastructure categories. The most critical problems include a broken WebSocket implementation (socket.io vs gorilla/websocket mismatch), a non-functional deployment pipeline (nil Docker client), severe frontend-backend API mismatches that prevent basic operations from working, hardcoded security credentials, and incomplete placeholder implementations for core features like database provisioning, SSL certificates, and cron jobs.

Table of Contents

1. Project Overview	3
1.1 Technology Stack	3
2. Critical Bugs	4
3. High Severity Issues	5
4. Security Vulnerabilities	6
5. Frontend-Backend API Mismatches	8
5.1 URL and Method Mismatches	8
5.2 Response Shape Mismatches	8
6. Incomplete and Placeholder Implementations	9
7. Infrastructure and Deployment Issues	10
8. Frontend Code Quality Issues	11
9. Missing Features	12
9.1 Completely Missing Features	12
10. Summary and Recommendations	13
10.1 Issue Summary	13
10.2 Priority Recommendations	13

1. Project Overview

Sofa is a self-hosted PaaS (Platform as a Service) panel designed to provide one-click deployment of web applications, similar to Vercel or Railway but running on your own server. The project features a Go backend with Gin and GORM, a React 19 frontend with TypeScript and Tailwind CSS 4, and integrates with PostgreSQL, Redis, Docker Engine, and Traefik for reverse proxying and auto-SSL. The stated features include auto framework detection, zero-downtime blue-green deployments, real-time log streaming, web terminal access, encrypted environment variables, custom domain support, database provisioning, resource monitoring, and deployment history with rollback capabilities.

However, after thorough analysis, it is clear that many of these features exist only as placeholder or stub implementations. The project has a well-structured codebase with clean separation between handler, service, repository, and infrastructure layers, but the actual business logic is largely incomplete. The deployment pipeline, which is the core feature of any PaaS, is non-functional due to a nil Docker client being passed to the pipeline. The WebSocket system has a fundamental protocol mismatch between the frontend (socket.io) and backend (gorilla/websocket), making real-time features entirely broken. These and other critical issues are detailed in the sections that follow.

1.1 Technology Stack

Layer	Technology	Version
Frontend	React + Vite + TypeScript + Tailwind CSS	React 19 / Vite 8
UI Components	Custom shadcn-style components	N/A
Backend	Go + Gin + GORM	Go 1.25 / Gin 1.12
Database	PostgreSQL (with SQLite fallback)	PostgreSQL 16
Cache/Queue	Redis + Asynq	Redis 7
Containers	Docker Engine API	Docker 28.5
Proxy	Traefik (auto-SSL)	Traefik v3
Real-time	gorilla/websocket (backend) / socket.io (frontend)	Mismatch

2. Critical Bugs

This section documents the most severe bugs found in the project. These issues prevent core functionality from working and must be fixed before the project can be considered functional. Each bug is categorized by severity: Critical means the feature is completely broken and cannot work; High means significant functionality is impaired; Medium indicates partial breakage or degraded behavior; Low represents minor issues or cosmetic problems.

ID	Severity	Category	Description
B01	Critical	WebSocket	Frontend uses socket.io-client but backend uses gorilla/websocket. These are incompatible protocols. socket.io has its own handshake, heartbeat, and packet framing. The backend /ws endpoint does raw WebSocket upgrade, so socket.io connections will always fail. All real-time features (logs, terminal, deployment status) are completely non-functional.
B02	Critical	Deploy Pipeline	The Docker client is initialized as nil in main.go (line 98: <code>deploy.NewPipeline(nil, ...)</code>). Since the <code>pipeline.dockerClient</code> is nil, any deployment attempt will panic with a nil pointer dereference. The entire deployment feature is completely broken.
B03	Critical	API Mismatch	Frontend uses slug-based routes (<code>appsApi.getApp(slug)</code> calls <code>GET /apps/{slug}</code>) but backend routes use numeric IDs (<code>apps.GET("/:id", ...)</code> with <code>getIDParam</code> parsing as <code>uint64</code>). Every single-app page, deployment, env var, and domain operation will return 400 Bad Request because the backend tries to parse the slug string as a number.
B04	Critical	API Mismatch	Frontend calls <code>PATCH /apps/{slug}</code> (<code>appsApi.updateApp</code> uses <code>.patch</code>) but backend registers <code>PUT /apps/:id</code> . The HTTP method mismatch means updates will return 405 Method Not Allowed.
B05	Critical	API Mismatch	Frontend <code>settingsApi.updateSettings</code> uses <code>PATCH /settings</code> but backend handler only registers <code>PUT /settings</code> . Settings updates will fail with 405 Method Not Allowed.
B06	Critical	API Mismatch	Frontend <code>databasesApi.provisionDB</code> sends <code>POST /databases</code> with <code>{name, type, appId}</code> but backend expects <code>POST /apps/:id/databases</code> with <code>{type, connection_string}</code> . The route, payload structure, and semantics are completely different.
B07	Critical	API Mismatch	Frontend <code>domainsApi.verifyDomain</code> calls <code>POST /apps/{slug}/domains/{id}/verify</code> but backend route is <code>POST /domains/:id/verify</code> . Different URL structure means verify never works.

B08	Critical	API Mismatch	Frontend domainsApi.setPrimary calls POST /apps/{slug}/domains/{id}/set-primary but backend route is POST /domains/:id/set-primary. Set-primary feature is broken.
B09	Critical	Env Vars	In pipeline.go line 109, env vars are passed as "ENCRYPTED" placeholder strings instead of being decrypted. Deployed containers receive literally "ENCRYPTED" as the value for every environment variable, making all deployments non-functional.
B10	Critical	Login Auth	In handler.go line 86, bcrypt.CompareHashAndPassword is called with adminPassword from config as the hash. But config stores the plaintext password "admin", not a bcrypt hash. This always fails. The fallback (line 88) allows direct plaintext comparison, which means bcrypt is effectively bypassed and passwords are compared in plaintext.

3. High Severity Issues

High severity issues represent significant functional problems that degrade the user experience or compromise the integrity of the system. While not completely fatal like critical bugs, these issues prevent important features from working correctly and should be prioritized for resolution.

ID	Severity	Category	Description
B11	High	Data Model	Frontend App type has fields like lastDeployAt, lastDeployCommit, url, startCommand that the backend model does not provide. The backend App model returns snake_case keys (source_type, source_url, cpu_limit, memory_limit) but the frontend expects camelCase. The response interceptor only unwraps the envelope; there is no automatic snake_case to camelCase conversion on the main app data.
B12	High	Data Model	Frontend Domain type expects {hostname, sslStatus, verified} but backend returns {domain, type, ssl_enabled, cert_expiry}. Field name mismatches mean domain display will show empty/undefined values everywhere.
B13	High	Data Model	Frontend Database type expects {name, status, host, port, username, password, connectionUrl, sizeMB, appName} but backend returns {type, container_id}. Almost no fields match, making the database page non-functional.
B14	High	Docker	Docker client is never initialized in main.go. There is no call to docker.NewDockerClient() anywhere in the startup code. The dockerClient variable remains nil throughout the application lifecycle, not just in the pipeline.

B1 5	High	Docker	In docker/client.go line 25, <code>os.Setenv("DOCKER_HOST", ...)</code> is called which mutates the process-wide environment. This is not goroutine-safe and can interfere with other components. Should use <code>client.WithHost()</code> option instead.
B1 6	High	Traefik	In proxy/traefik.go line 209, the <code>updateDynamicConfig</code> method sends PUT to <code>/api/providers/rest</code> . This endpoint does not exist in Traefik v3. Traefik v3 removed the REST provider. The dynamic configuration must be provided via Docker labels (which the code partially does) or file-based providers. This API call will always return 404.
B1 7	High	Deploy	<code>Builder.Build()</code> in <code>builder.go</code> tries to build from <code>/tmp/sofa-build-{slug}</code> for Dockerfile sources, but this directory is never populated with source code. Only git sources actually clone to a temp directory. Dockerfile deployments are non-functional.
B1 8	High	Config	Viper env binding with nested keys does not work correctly by default. <code>SOFA_DATABASE_HOST</code> maps to <code>database.host</code> only if <code>viper.BindEnv()</code> is called explicitly. With just <code>AutomaticEnv()</code> , the nested struct unmarshaling from environment variables will likely fail, meaning Docker container environment variables may not be properly read.
B1 9	High	Auth	There is no token refresh mechanism. JWT tokens expire after the configured duration (default 24h) with no way to renew them. After expiration, users must re-login. There is also no token revocation/blacklist, so compromised tokens remain valid until expiry.
B2 0	High	Security	The <code>validateJWT</code> function in <code>handler.go</code> (line 1088-1090) returns <code>nil</code> , <code>nil</code> - it is a dead stub. While it is currently unused (auth uses <code>middleware.ValidateToken</code> instead), its presence suggests incomplete refactoring and could be accidentally used in the future.

4. Security Vulnerabilities

Security is a critical concern for any PaaS platform that manages container deployments and handles sensitive data like environment variables and database credentials. The Sofa project contains several significant security vulnerabilities that must be addressed before any production deployment. These range from hardcoded credentials to inadequate authentication mechanisms and CORS configurations that are too permissive.

ID	Severity	Category	Description
----	----------	----------	-------------

B2 1	Critical	Credentials	Hardcoded credentials throughout the project: config.yaml has JWT secret "sofa-secret-change-in-production" and admin password "admin". docker-compose.yml has "sofa-production-secret-change-me" as JWT secret and "admin" as password. These are committed to the repository and would be used in production if not manually changed.
B2 2	High	CORS	The CORS middleware (middleware.go) allows any origin (reflects the Origin header back). For a self-hosted panel, this means any website can make cross-origin requests to the API if the user has a valid token. Combined with no CSRF protection, this enables cross-site request forgery attacks.
B2 3	High	WebSocket	The WebSocket upgrader (realtime/hub.go) allows all origins (CheckOrigin returns true). No authentication is required to connect to /ws. Any client can connect, subscribe to deployment rooms, and receive real-time events including build logs that may contain sensitive information.
B2 4	High	Auth	The login endpoint accepts the admin password in plaintext and compares it directly (handler.go line 88). There is no rate limiting on login attempts, making brute-force attacks trivial. No account lockout mechanism exists after failed attempts.
B2 5	Medium	Encryption	The encryption key for environment variables is derived from the JWT secret (crypto.go). This means anyone who knows the JWT secret can decrypt all stored environment variables. The key should be independently generated and stored separately.
B2 6	Medium	Input Validation	The UpdateApp handler accepts arbitrary map[string]interface{} and applies it to the app model (service.go lines 122-145). While it checks specific allowed fields, there is no validation of values. A malicious user could set port to a privileged port, cpu_limit to an extreme value, or memory_limit to 0.
B2 7	Medium	Git Clone	The git clone operation (git/operations.go) supports only public repositories and basic auth. SSH key authentication is commented out. Clone URLs are not validated, which could allow SSRF attacks if an attacker provides a URL pointing to internal services.
B2 8	Low	Docker Socket	The Docker socket is mounted into both the backend container and the Traefik container in docker-compose.yml. This gives both containers full root access to the host Docker daemon. A compromise of either container means full host compromise. The socket should be mounted read-only where possible.

5. Frontend-Backend API Mismatches

The most pervasive class of bugs in the Sofa project is the disconnect between what the frontend expects and what the backend provides. These mismatches go beyond simple naming conventions and extend to fundamental differences in URL structure, HTTP methods, request payloads, and response shapes. The frontend was clearly designed against an idealized API specification that the backend does not implement. This section provides a comprehensive mapping of all identified discrepancies.

5.1 URL and Method Mismatches

ID	Severity	Category	Description
B29	High	API Route	GET /apps/{slug} - Frontend sends slug string, backend parses as uint64 ID. Every app detail page fails.
B30	High	API Route	PATCH /apps/{slug} vs PUT /apps/:id - Different method AND different parameter type. App updates are broken.
B31	High	API Route	DELETE /apps/{slug} - Backend uses numeric ID. Deleting apps by slug fails.
B32	High	API Route	POST /apps/{slug}/start stop restart - Backend uses numeric ID. Start/stop/restart operations fail.
B33	High	API Route	GET /apps/{slug}/deployments - Backend uses numeric ID. Listing deployments fails.
B34	High	API Route	POST /databases - Frontend posts to global endpoint, backend expects POST /apps/:id/databases. Database provisioning is broken.
B35	High	API Route	PATCH /settings vs PUT /settings - Method mismatch. Settings updates fail with 405.
B36	Medium	API Route	GET /metrics/apps/{slug} - Backend route uses numeric ID /metrics/apps/:id. App metrics fail.

5.2 Response Shape Mismatches

Even when API calls succeed, the response data often does not match what the frontend expects. The backend returns snake_case keys (source_type, cpu_limit, memory_limit) while the TypeScript types define camelCase fields (sourceType, cpuLimit, memoryLimit). The only conversion happens in the snakeToCamel utility function, but it is only applied to metrics and server stats responses, not to the main app, deployment, domain, or database data. This means most data displayed in the UI will show as undefined or incorrectly parsed.

Furthermore, the backend App model returns fields like `source_type`, `build_cmd`, `container_id`, `image_id` that the frontend App type does not have. Conversely, the frontend expects fields like `lastDeployAt`, `lastDeployCommit`, `url`, `startCommand` that the backend never returns. The backend Domain model returns "domain" (the hostname), "type", "ssl_enabled" while the frontend expects "hostname", "sslStatus", "verified". The backend Database model returns only "type" and "container_id" while the frontend expects a full connection info object with name, host, port, username, password, and `connectionUrl`. These mismatches mean that even if the API routes were correct, the data would not render properly in the UI.

6. Incomplete and Placeholder Implementations

A significant portion of the Sofa codebase consists of stub or placeholder implementations that do not perform any real work. These are typically marked with comments like "In production, this would..." but they represent features that are advertised as functional in the README. Users who deploy this project expecting these features to work will be disappointed. Below is a comprehensive inventory of every identified placeholder.

ID	Severity	Category	Description
B37	High	Deploy Pipeline	The deploy pipeline <code>RunPipeline()</code> cannot execute because <code>dockerClient</code> is nil. Even if fixed, the health check only checks if the container is running (pipeline.go line 198-204), with no HTTP health endpoint check. The comment says "In production, we would make an HTTP request."
B38	High	DB Provision	The <code>handleDBProvision</code> worker (queue/worker.go lines 233-248) is a complete no-op. It logs the task and returns nil without creating any Docker container, waiting for readiness, or updating connection strings. Database provisioning is entirely non-functional.
B39	High	SSL Certs	The <code>handleSSLCertificate</code> worker (queue/worker.go lines 250-264) is a complete no-op. SSL certificate provisioning via Let's Encrypt / ACME is not implemented. Domains will never have SSL certificates auto-provisioned.
B40	High	Cron Jobs	The <code>handleCronJob</code> worker (queue/worker.go lines 266-292) only updates the <code>last_run</code> timestamp. It does not execute the command in the app container. Cron job execution is non-functional.
B41	High	Domain Verify	<code>DomainService.VerifyDomain</code> (service.go line 376) returns nil without doing anything. DNS verification is not implemented. <code>DomainService.SetPrimary</code> (line 381) also returns nil without making any changes.

B4 2	High	Metrics	GetAppMetrics (handler.go lines 749-773) returns all zeros for CPU, memory, network, and uptime. The comment says "in production these would come from Docker stats." The GetContainerStats method in docker/client.go (line 170) also discards the actual data with "_ = data".
B4 3	High	Container Stats	DockerClient.GetContainerStats reads the full stats response but then ignores it with "_ = data" (docker/client.go line 170). All stat values are returned as zero. This makes resource monitoring completely non-functional.
B4 4	Medium	Activity Feed	The /api/v1/activity endpoint (handler.go line 1071) returns an empty array. No activity tracking is implemented anywhere in the codebase. The dashboard activity feed will always be empty.
B4 5	Medium	SSH Key Auth	SSH key authentication for git clone is commented out in git/operations.go (lines 34-40). Only public repos work. The SSH key management UI exists but the keys are never used during cloning.
B4 6	Medium	Remove Routing	TraefikManager.RemoveAppRouting (proxy/traefik.go lines 113-119) is a no-op. It constructs a name string but never uses it. Removing app routing does nothing.

7. Infrastructure and Deployment Issues

Beyond the application code itself, the project has several infrastructure-level problems that affect deployment reliability, security, and operational correctness. These include Docker configuration issues, missing health checks, port conflicts, and database migration concerns.

ID	Severity	Category	Description
B4 7	High	Docker Compose	The frontend service (port 7632:80) and Traefik (port 8081:80) both expose HTTP on port 80 internally, but the README says to access the panel at localhost:3000. No service listens on port 3000. Users cannot access the application as documented.
B4 8	Medium	Docker Compose	The backend Dockerfile uses golang:1.23-alpine but go.mod specifies go 1.25.0. While GOTOOLCHAIN=auto is set, this adds unnecessary download overhead during build. The base image should match the go.mod version.
B4 9	Medium	Docker Compose	Traefik is configured to redirect HTTP to HTTPS (lines 51-53 of docker-compose.yml) and uses LetsEncrypt TLS challenge. But in local development, this makes the panel inaccessible because LetsEncrypt cannot issue certificates for localhost. The redirect should be conditional.

B5 0	Medium	Database	The project has SQL migration files (migrations/000001_init_schema.up.sql) but they are never used. main.go uses GORM AutoMigrate which can silently alter schemas. Running migrations manually would conflict with AutoMigrate. There is no migration runner.
B5 1	Low	Docker Compose	The docker-compose.yml does not set GIN_MODE=release for the backend. The main.go defaults to ReleaseMode when GIN_MODE is empty (line 146), but this is counterintuitive. The Dockerfile also does not set it explicitly.

8. Frontend Code Quality Issues

The frontend codebase, while visually well-designed with a dark theme and professional UI components, contains several code quality issues that affect maintainability and correctness. ESLint analysis revealed multiple warnings and errors, and there are architectural concerns with how data flows between the frontend and backend.

ID	Severity	Category	Description
B5 2	Medium	ESLint	Multiple unused import errors: Button in header.tsx, CardHeader/CardTitle in app-deployments.tsx, app-env.tsx, domains.tsx, cn in app-deployments.tsx, toast in app-settings.tsx. Dead imports increase bundle size and indicate incomplete refactoring.
B5 3	Medium	React Anti-Pattern	In app-settings.tsx line 32, setState (setName) is called synchronously within useEffect. React docs explicitly warn against this as it causes cascading renders. Should use a derived state pattern or compute during render.
B5 4	Low	Bundle Size	The production build produces a single 1008 KB JavaScript chunk (exceeds 500 KB limit). Code splitting with React.lazy() and dynamic imports should be used for route-level splitting to improve initial load time.
B5 5	Low	Type Safety	Multiple ESLint no-explicit-any errors in UI components (dialog.tsx, dropdown-menu.tsx). While these are in shadcn-style component files, they reduce type safety and should use proper generic types.
B5 6	Low	Socket.io	The socket.io-client import adds ~40 KB to the bundle but is completely non-functional due to the backend protocol mismatch. This dead code should be removed or replaced with a native WebSocket client.

9. Missing Features

The README and project description advertise a number of features that are either completely missing or exist only as empty data structures in the database schema. The following is a comprehensive list of features that are claimed but not actually implemented in the codebase. These represent a significant gap between the project description and its actual capabilities.

9.1 Completely Missing Features

Feature	Current Status
Auto Framework Detection	DetectFramework() exists in builder.go but is only called during build, not at app creation. The frontend DeployWizard does not show detected frameworks. The detection logic is basic file-existence checks with no package.json parsing.
Blue-Green Deployments	The pipeline stops the old container after the new one is healthy, but there is no period where both are running simultaneously. True blue-green deployment requires both containers to be up with traffic switching, which is not implemented.
Real-time Log Streaming	The WebSocket hub broadcasts events, but there is no mechanism to stream Docker container logs in real-time. The GetContainerLogs method returns all logs at once, not as a stream. The xterm.js terminal component exists but has no backend endpoint.
Web Terminal Access	The frontend has a web-terminal.tsx component using xterm.js, but there is no corresponding backend endpoint to create an exec session and stream I/O. The Docker client has ExecContainer() but it is never wired to a handler.
Database Provisioning	The handleDBProvision worker is a no-op. No Docker containers are created for databases. The frontend databases page shows a UI but no actual databases can be provisioned.
Resource Monitoring	All metrics return zero. CPU, memory, network, and uptime data is not collected from Docker or the system. The monitoring UI displays empty/zero values.
Cron Job Execution	Cron jobs are stored in the database but never executed. There is no scheduler that reads cron expressions and triggers tasks. The handleCronJob worker only updates timestamps.
SSL Certificate Management	The handleSSLCertificate worker is a no-op. Let's Encrypt integration is not implemented. Traefik handles TLS termination but Sofa never requests or manages certificates.

10. Summary and Recommendations

10.1 Issue Summary

Severity	Count	Description
Critical	11	Core functionality completely broken (WebSocket, deploy pipeline, API mismatches, auth)
High	19	Significant features impaired (data model mismatches, security, placeholder implementations)
Medium	13	Partial breakage or quality issues (CORS, encryption key, bundle size, database migrations)
Low	5	Minor issues (unused imports, bundle size, Docker socket, GIN_MODE)
Total	48+	Combined issues across all severity levels

10.2 Priority Recommendations

1. Fix the WebSocket Protocol Mismatch (Critical): Replace socket.io-client on the frontend with a native WebSocket client that matches the gorilla/websocket backend, or replace the backend with a socket.io-compatible server. This is the single most impactful fix because real-time features are central to the PaaS experience.

2. Initialize the Docker Client (Critical): Add `docker.NewDockerClient()` call in `main.go` and pass the initialized client to the pipeline. Without this, deployments are impossible. Consider making Docker optional with graceful degradation when unavailable.

3. Unify the API Contract (Critical): Decide whether routes use slug or ID, then make both frontend and backend consistent. Add a comprehensive API specification (OpenAPI/Swagger) that both sides can validate against. Implement consistent snake_case to camelCase conversion in the response interceptor.

4. Implement Missing Core Features (High): Focus on database provisioning, SSL certificate management, cron job execution, and real-time log streaming. These are the features users expect from a PaaS and are currently stubs.

5. Fix Security Vulnerabilities (High): Remove hardcoded credentials, implement bcrypt password hashing properly, add rate limiting on login, restrict CORS origins, add WebSocket authentication, and generate a separate encryption key for environment variables.

6. Add Integration Tests (High): The project has zero tests. Add integration tests that verify the frontend can communicate with the backend for all major operations. This would have caught most of the API mismatch bugs immediately.

7. Fix the Frontend Build (Medium): Implement code splitting for route-level components, remove unused imports, fix the setState-in-effect anti-pattern, and replace the non-functional socket.io-client with a working WebSocket implementation.

8. Documentation and Configuration (Medium): Fix the README to reflect the actual access port (7632, not 3000). Add environment variable documentation. Create a CONTRIBUTING.md with development setup instructions. Add an OpenAPI specification for the API.

In conclusion, the Sofa project has a promising architecture and clean code organization, but it is far from a functional PaaS panel. The critical issues, particularly the WebSocket mismatch, nil Docker client, and pervasive API inconsistencies, mean that the application cannot perform its core function of deploying applications. The project needs focused effort on making the existing features actually work before adding new ones. A recommended approach would be to fix the critical bugs first, then implement integration tests, and finally work through the high and medium severity issues in priority order.